

25CS1302E – DBMS

WEEK–10 COURSEWORK

FastAPI HTTP Methods – Student Management API

Student: Talluri Hima Bindu

Roll No: 2520030484

Experiment	FastAPI HTTP Methods
Application	Student Management API
Framework	FastAPI
Server	Uvicorn – http://127.0.0.1:8000

1. Objective

To develop and test a practical FastAPI-based Student Management API demonstrating the major HTTP methods GET, POST, PUT, PATCH and DELETE using Swagger UI.

2. API Endpoints Tested

Method	Endpoint	Purpose
GET	/	Home / API welcome
GET	/students	Retrieve all students
POST	/students	Create a student
GET	/students/{student_id}	Retrieve a specific student
PUT	/students/{student_id}	Completely update a student
PATCH	/students/{student_id}	Partially update a student
DELETE	/students/{student_id}	Delete a student

3. Testing and Execution Evidence

The following screenshots document the successful execution of the API operations in Swagger UI.

1. FastAPI Swagger UI – Student Management API

Student Management API 1.0.0 OAS 3.1

/openapi.json

A practical FastAPI project demonstrating HTTP methods

default

GET	/	Home	⌵
GET	/students	Get Students	⌵
POST	/students	Create Student	⌵
GET	/students/{student_id}	Get Student	⌵
PUT	/students/{student_id}	Update Student	⌵
PATCH	/students/{student_id}	Partial Update Student	⌵
DELETE	/students/{student_id}	Delete Student	⌵

Schemas

HTTPValidationError > Expand all object

2. POST /students – Create Student request

POST /students Create Student

Parameters

Cancel

Reset

Name	Description
student_id * required integer (query)	101

Request body required

application/json

Edit Value | Schema

```
{
  "name": "Rahul",
  "age": 20,
  "course": "Computer Science",
  "email": "rahul@example.com"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/students?student_id=101' \
  -H 'accept: */*' \
  -H 'content-type: application/json' \
  -d '{
    "name": "Rahul",
    "age": 20,
    "course": "Computer Science",
    "email": "rahul@example.com"
  }'
```

Request URL

```
http://127.0.0.1:8000/students?student_id=101
```

3. GET /students – Retrieve all students

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** /students
- Parameters:** No parameters
- Execute:** Button to execute the request
- Clear:** Button to clear the request
- Responses:** Section showing the response details
- Curl:**

```
curl -X 'GET' \
'http://127.0.0.1:8000/students' \
-H 'accept: */*'
```
- Request URL:** http://127.0.0.1:8000/students
- Server response:** 200
- Response body:**

```
{
  "students": {
    "101": {
      "name": "Rahul",
      "age": 20,
      "course": "Computer Science",
      "email": "rahul@example.com"
    }
  }
}
```
- Response headers:** (Empty)

4. GET /students/{student_id} – Retrieve student 101

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://127.0.0.1:8000/docs#/default/get_student_students_student_id_get
- Request URL:** http://127.0.0.1:8000/students/101
- Server response:** 200
- Response body:**

```
{
  "name": "Rahul",
  "age": 20,
  "course": "Computer Science",
  "email": "rahul@example.com"
}
```
- Response headers:**

```
content-length: 81
content-type: application/json
date: Thu, 24 Sep 2020 05:08:54 GMT
server: uvicorn
```
- Responses:** Table showing the response details

Code	Description	Links
200	Successful Response	No links
422	Validation Error	No links

The table also includes a 'Media type' dropdown set to 'application/json' and an 'Example Value' field with the value 'string'.

5. PUT /students/{student_id} – Complete update of student 101

Responses

Curl

```
curl -X 'PUT' \
  'http://127.0.0.1:8000/students/101' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Rahul Kumar",
    "age": 21,
    "course": "Computer Science and Engineering",
    "email": "rahulkumar@example.com"
  }'
```

Request URL

http://127.0.0.1:8000/students/101

Server response

Code	Details
200	Response body <pre>{ "message": "Student completely updated", "student_id": 101, "student": { "name": "Rahul Kumar", "age": 21, "course": "Computer Science and Engineering", "email": "rahulkumar@example.com" } }</pre> Response headers <pre>content-length: 176 content-type: application/json date: Thu, 24 Sep 2026 05:10:09 GMT server: uvicorn</pre>

Responses

Code	Description	Links
------	-------------	-------

6. PATCH /students/{student_id} – Partial update of student 101

Responses

Curl

```
curl -X 'PATCH' \
  'http://127.0.0.1:8000/students/101' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "string",
    "age": 0,
    "course": "string",
    "email": "user@example.com"
  }'
```

Request URL

http://127.0.0.1:8000/students/101

Server response

Code	Details
200	Response body <pre>{ "message": "Student partially updated", "student_id": 101, "student": { "name": "string", "age": 0, "course": "string", "email": "user@example.com" } }</pre> Response headers <pre>content-length: 137 content-type: application/json date: Thu, 24 Sep 2026 05:11:27 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

7. DELETE /students/{student_id} – Delete student 101

Name	Description
student_id * required integer (path)	101

Execute

Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://127.0.0.1:8000/students/101' \
  -H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8000/students/101
```

Server response

Code	Details
200	Response body <pre>{ "message": "Student deleted successfully", "student_id": 101, "student": { "name": "String", "age": 0, "course": "string", "email": "user@example.com" } }</pre>

8. Final GET /students – Empty collection after deletion

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/students' \
  -H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8000/students
```

Server response

Code	Details
200	Response body <pre>{ "students": {} }</pre> Response headers <pre>content-length: 15 content-type: application/json date: Thu, 24 Sep 2026 05:19:16 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header. Example Value Schema <pre>"string"</pre>	No links

4. Result

The Student Management API was successfully executed using FastAPI and Uvicorn. The GET, POST, PUT, PATCH and DELETE HTTP methods were tested through Swagger UI. Student ID 101 was created, retrieved, updated, partially updated and finally deleted. The final GET /students request returned an empty students object, confirming the deletion.

5. Conclusion

The experiment demonstrates how FastAPI can be used to build RESTful APIs and handle different HTTP methods for student data management. Swagger UI provided an interactive interface for testing the API endpoints and observing their responses.